

Report
Of
Real-Time Stock Market Dashboard

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By

Name of Student: ADITYA RAJ

Roll Number : 25SCS1003002468

Section: 2CSE33

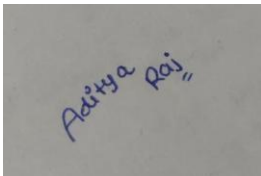
Batch: 2025-29

Candidate's Declaration

I, Aditya Raj, do hereby declare that the internship report titled “Real-Time Stock Market Dashboard” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement, and further I shall be solely responsible for any kind of copyright violation in this regard.

This report reflects the work carried out entirely during the one-month internship tenure, and all code, screenshots, and outcomes described herein were produced and verified by me as part of the assigned project.

Signature: -

A photograph of a piece of paper with the name "Aditya Raj" written in blue ink in a cursive, handwritten style.

Student Name: ADITYA RAJ

Roll No.: 25SCS1003002468

Acknowledgement

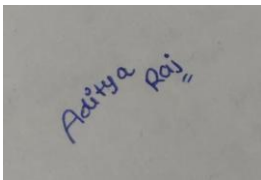
I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to sincerely thank Codec Technologies Pvt. Ltd. for providing me the opportunity to work as a Python Developer Intern and for the practical exposure gained during this internship.

I express my gratitude to my parents for their blessings.

I would also like to thank the faculty of the School of Computer Science and Engineering, IILM University, for their continuous academic support and for encouraging students to pursue industry internships alongside coursework, which made it possible to translate classroom concepts into a working, real-world application.

Signature:-

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature reads "Aditya Raj" in a cursive, slanted script.

Student Name: ADITYA RAJ
Roll No.: 25SCS1003002468

Internship Completion Certificate



Certificate of Internship — National Internship Portal (AICTE, Ministry of Education), issued by Codec Technologies Pvt. Ltd.



Fig — Certificate of Completion — Python Development Training Program (ICAC recognized)

Both certificates were issued directly by Codec Technologies Pvt. Ltd. on successful completion of the one-month internship and are independently verifiable via the QR codes and NCS ID printed on each certificate.

Table of Contents

Chapter No.	Chapter Name	Page Nos.
	Cover Page.....	—
1.	Candidate's Declaration	ii
2.	Acknowledgement	iii
3.	Internship Completion Certificate.....	iv
4.	Project Description.....	6
	4.1 Introduction	6
	4.2 Organization Profile	6
	4.3 Problem Statement	6
	4.4 Project Objectives	7
	4.5 Scope of the Project.....	7
	4.6 Technologies and Tools Used	7
	4.7 System Architecture.....	8
	4.8 Methodology.....	8
	4.9 Expected Outcomes	8
	4.10 Code Implementation (app.py)	9
	4.11 Dashboard Screenshots (Output).....	13
	4.12 Certificates of Completion and Other Communication / Mail Proof.....	15
	4.13 Challenges Faced and Key Learnings	17
	4.14 Conclusion.....	17
5.	Bibliography / References	18

4. Project Description

4.1 Introduction

This report presents the work carried out during my internship as a Python Developer Intern at Codec Technologies Pvt. Ltd., undertaken as part of the B.Tech Computer Science Engineering programme (Semester 3). The internship provided an opportunity to move beyond classroom learning and gain hands-on, practical exposure to real-world software development using Python.

The core deliverable of the internship was the design and development of the Real-Time Stock Market Dashboard, a Python-based interactive web application that allows users to monitor stock market information. Through this project, I was able to apply concepts of data retrieval, data processing, data visualization, and web application development in a single, cohesive product.

The internship was carried out over the period from 08/06/2026 to 08/07/2026, during which I worked under the guidance of the organization's technical team to plan, build, test, and document the application.

Working on this project also gave me an appreciation for the full lifecycle of a small software product — from gathering requirements and choosing the right libraries, through iterative development and testing, to finally documenting the work in a form that is presentable to both a technical and non-technical audience. This report is organized to walk through that same journey: it begins with the problem being solved, moves through the design and technology choices, and closes with the working outcome, supporting evidence, and reflections on what was learned.

4.2 Organization Profile

Codec Technologies Pvt. Ltd. is a technology-oriented organization that provides learning and development opportunities with practical exposure to programming and software development. It delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth, and offers internship programs designed to help students strengthen their technical skills by working on real, application-oriented projects under mentorship.

The internship was offered under the designation of Python Developer Intern, in Pan India / Hybrid mode, for a duration of one month (08/06/2026 to 08/07/2026), reporting to the assigned Project Head(s). Codec Technologies is a Google for Education Partner and an ISO 9001:2015 certified organization, and the program is recognized on the AICTE National Internship Portal.

As part of its internship model, Codec Technologies structures the program around self-paced project tasks combined with periodic performance reviews, which allows interns to learn primarily through building a working deliverable rather than through passive instruction. This project-first approach was well suited to developing the Real-Time Stock Market Dashboard, since it required continuous experimentation with libraries such as yfinance and Plotly rather than following a fixed, pre-defined syllabus.

4.3 Problem Statement

Stock market information is often scattered across different sources, making it difficult for an ordinary user to interpret it quickly and confidently. Investors and enthusiasts typically need to visit multiple websites or applications to check a stock's current price, its historical performance, its trading volume, and general company information.

There was, therefore, a need for a simple, unified interface where a user could select a stock of interest and view its important current information, explore historical price trends, and monitor trading activity — all from a single dashboard.

This fragmentation is not just an inconvenience — it can slow down decision-making for someone trying to quickly gauge whether a stock is trending up or down, or how volatile it has been recently. A single-screen dashboard that consolidates price, trend, volume, and company context directly addresses this by removing the need to cross-reference multiple sources for a single decision.

4.4 Project Objectives

- To provide an interactive stock monitoring interface for users.
- To retrieve real-time and historical stock market data programmatically.
- To display current market information such as price, price change, and day high/low.
- To visualize historical price trends using interactive charts.
- To display trading volume over a selected time period.
- To provide relevant company information alongside market data.
- To make financial data easier to understand for a general user through visualization.
- To keep the interface responsive and readable through a wide, single-page layout that works well on both laptop and desktop screens.
- To handle missing or unavailable data (e.g., no market cap, no trading data) gracefully instead of letting the application crash.

4.5 Scope of the Project

The scope of the project is limited to building a functional, single-user web dashboard for viewing publicly available stock market data for selected companies (such as Apple and NVIDIA). The application allows a user to select a stock and a time period, and view the corresponding price trends, trading volume, recent market data, and company information through an interactive Streamlit-based interface.

The project does not include real-money trading, order placement, user account management, or portfolio management — these are identified as areas of future enhancement and are discussed further under Expected Outcomes and Future Scope.

Within this scope, the dashboard supports seven pre-configured stocks (Apple, Microsoft, Google, Amazon, Tesla, NVIDIA, and Meta) and six selectable time periods (1 month, 3 months, 6 months, 1 year, 2 years, and 5 years), which was considered sufficient to demonstrate the core functionality without expanding the project beyond what could be reliably built, tested, and documented within the one-month internship window.

4.6 Technologies and Tools Used

- Python — core programming language used to build the entire application.
- Streamlit — framework used to create the interactive web dashboard interface.
- Pandas — used for structuring and processing stock market data.
- Plotly — used to create interactive charts and visualizations.
- yfinance — Python library used to retrieve stock market data.
- Visual Studio Code — primary code editor used for development.
- GitHub — used for version control and project submission.

Streamlit was chosen over a traditional Flask/Django plus HTML/CSS/JavaScript stack because it allows a data-focused web interface to be built almost entirely in Python, which sped up development considerably within the internship timeframe. Similarly, Plotly was preferred over static plotting libraries such as Matplotlib because its charts are interactive by default (zoom, hover tooltips, pan), which suits a dashboard meant for exploration rather than a fixed report figure.

4.7 System Architecture

The application follows a simple, linear data-flow architecture suited to a single-page interactive dashboard. The user interacts with the Streamlit front end to select a stock and a time period; this selection triggers a data-retrieval call through the `yfinance` library, which fetches market data from its underlying data source. The retrieved data is then processed and structured using `Pandas`, after which `Plotly` is used to render interactive charts (historical price and trading volume). The processed data and visualizations are finally rendered back to the user through the Streamlit interface.

Workflow: User → Select Stock → Select Time Period → Retrieve Market Data (yfinance) → Process Data (Pandas) → Generate Charts (Plotly) → Display Interactive Dashboard (Streamlit)

Two additional details support this architecture in practice. First, data retrieval is wrapped in a cached function (a five-minute time-to-live cache via `st.cache_data`) so that repeated interactions with the same stock and period do not trigger unnecessary network calls, keeping the dashboard responsive. Second, the entire retrieval-and-render flow is wrapped in a `try/except` block, so that if the data source is temporarily unavailable or a symbol returns no data, the user sees a clear on-screen message instead of the application crashing.

4.8 Methodology

The project was developed using an iterative approach. The first phase involved understanding the requirements and identifying the core features to be built. In the second phase, the basic Streamlit interface was created along with stock selection and time-period controls. In the third phase, data retrieval was implemented using the `yfinance` library, followed by data processing with `Pandas` to structure the retrieved information.

In the following phase, interactive visualizations — including historical price charts and trading volume charts — were built using `Plotly` and integrated into the dashboard. Company information and a recent market data table were then added to complete the feature set. Finally, the application was tested feature-by-feature and refined based on the results of this testing.

Testing at each phase was largely manual: after adding a feature, it was checked against multiple stocks (e.g., Apple and NVIDIA) and multiple time periods to confirm the charts, metrics, and tables updated correctly and that edge cases — such as a stock with fewer historical data points — did not break the layout. Code was committed to GitHub incrementally as each feature was completed, which made it easy to track progress and roll back if a change introduced an issue.

4.9 Expected Outcomes

By the end of the internship, the Real-Time Stock Market Dashboard was expected to, and did, successfully provide the following outcomes:

- An interactive dashboard allowing users to select a stock (e.g., Apple, NVIDIA) and a time period.
- Display of current stock price along with price change and percentage change.

- Display of day high and day low values.
- Interactive historical price visualization for the selected stock and period.
- Interactive trading volume visualization.
- A recent market data table for quick reference.
- Display of relevant company information.

Beyond the application itself, the internship helped me gain practical experience in Python development, working with external APIs and data, data processing with Pandas, building interactive visualizations with Plotly, developing web applications with Streamlit, debugging and testing, using GitHub for version control, writing technical documentation, and presenting project work.

Future Scope: the dashboard can be further extended by adding more financial indicators, user authentication, watchlists, price alerts, portfolio tracking, and improved dashboard analytics.

In addition to the future scope noted above, the dashboard could be enhanced with technical indicators such as moving averages, RSI, or MACD for more advanced trend analysis, a comparison view that overlays two or more stocks on the same chart, and deployment to a cloud host (such as Streamlit Community Cloud) so that the dashboard is accessible via a public URL rather than only running locally.

4.10 Code Implementation (app.py)

The following screenshots show the complete source code of app.py, the core script that implements the Streamlit dashboard, data retrieval, processing, and visualization logic described above.

The script is organized into clearly commented sections — page configuration, sidebar controls, data loading, current market metrics, the price chart, the volume chart, the market data table, and company information — which keeps each concern isolated and makes the ~226-line file easy to navigate despite covering the entire application in a single file.

```

app.py > ...
1 import streamlit as st
2 import yfinance as yf
3 import pandas as pd
4 import plotly.graph_objects as go
5
6 # -----
7 # PAGE CONFIGURATION
8 # -----
9 st.set_page_config(
10     page_title="Stock Market Dashboard",
11     page_icon="📈",
12     layout="wide"
13 )
14
15 # -----
16 # TITLE
17 # -----
18 st.title("📈 Real-Time Stock Market Dashboard")
19 st.markdown(
20     "Monitor stock prices, historical trends, trading volume, "
21     "and basic market indicators."
22 )
23
24 # -----
25 # SIDEBAR
26 # -----
27 st.sidebar.header("Stock Selection")
28
29 stock_options = {
30     "Apple": "AAPL",
31     "Microsoft": "MSFT",
32     "Google": "GOOGL",
33     "Amazon": "AMZN",
34     "Tesla": "TSLA",
35     "NVIDIA": "NVDA",
36     "Meta": "META"
37 }
38
39 selected_stock = st.sidebar.selectbox(
40     "Select a Stock",
41     list(stock_options.keys())
42 )
43
44 ticker_symbol = stock_options[selected_stock]
45
46 period = st.sidebar.selectbox(
47     "Select Time Period",
48     ["1mo", "3mo", "6mo", "1y", "2y", "5y"]
49 )
50

```

Fig 4.10.1 — Page configuration, title, and sidebar stock/time-period selection (lines 1–50)

This section sets the page title and a wide layout, defines the dictionary mapping friendly company names to their ticker symbols, and builds the two sidebar controls (stock selector and time-period selector) that drive every other part of the dashboard.

```

app.py > ...
51 # -----
52 # LOAD STOCK DATA
53 # -----
54 @st.cache_data(ttl=300)
55 def get_stock_data(symbol, selected_period):
56     ticker = yf.Ticker(symbol)
57     data = ticker.history(period=selected_period)
58     info = ticker.info
59     return data, info
60
61
62 try:
63     data, info = get_stock_data(ticker_symbol, period)
64
65     if data.empty:
66         st.error("No stock data available. Please try again.")
67         st.stop()
68
69     # -----
70     # CURRENT MARKET INFORMATION
71     # -----
72     current_price = float(data["Close"].iloc[-1])
73
74     previous_close = (
75         float(data["Close"].iloc[-2])
76         if len(data) > 1
77         else current_price
78     )
79
80     price_change = current_price - previous_close
81
82     percentage_change = (
83         (price_change / previous_close) * 100
84         if previous_close != 0
85         else 0
86     )
87
88     day_high = float(data["High"].iloc[-1])
89     day_low = float(data["Low"].iloc[-1])
90     volume = int(data["Volume"].iloc[-1])
91

```

Fig 4.10.2 — Cached stock-data retrieval and current market metric calculations (lines 51–91)

The `get_stock_data` function is decorated with `@st.cache_data(ttl=300)` so results are reused for five minutes, and it returns both historical price data and ticker info in one call. The current price, previous close, price change, percentage change, day high, day low, and volume are then derived from this data with simple safeguards against division by zero.

```

93 # METRIC CARDS
94 # -----
95 st.subheader(f"({selected_stock}) ({ticker_symbol})")
96
97 col1, col2, col3, col4 = st.columns(4)
98
99 col1.metric(
100     "Current Price",
101     f"${current_price:,.2f}",
102     f"${price_change:+.2f}"
103 )
104
105 col2.metric(
106     "Change %",
107     f"${percentage_change:+.2f}%"
108 )
109
110 col3.metric(
111     "Day High",
112     f"${day_high:,.2f}"
113 )
114
115 col4.metric(
116     "Day Low",
117     f"${day_low:,.2f}"
118 )
119
120 # -----
121 # PRICE CHART
122 # -----
123 st.subheader("📈 Historical Price")
124
125 fig = go.Figure()
126
127 fig.add_trace(
128     go.Scatter(
129         x=data.index,
130         y=data["Close"],
131         mode="lines",
132         name="Closing Price"
133     )
134 )
135
136 fig.update_layout(
137     title=f"({selected_stock}) Stock Price",
138     xaxis_title="Date",
139     yaxis_title="Price (USD)",
140     hovermode="x unified",
141     height=500
142 )
143
144 st.plotly_chart(fig, use_container_width=True)
145

```

Fig 4.10.3 — Metric cards and historical price chart using Plotly (lines 93–145)

Four metric cards (Current Price, Change %, Day High, Day Low) are laid out using `st.columns(4)`, and the historical price chart is built as a Plotly line trace with the date on the x-axis and closing price on the y-axis, using unified hover mode so all values at a given date appear together on hover.

```

147 # VOLUME CHART
148 # -----
149 st.subheader("📊 Trading Volume")
150
151 volume_fig = go.Figure()
152
153 volume_fig.add_trace(
154     go.Bar(
155         x=data.index,
156         y=data["Volume"],
157         name="Volume"
158     )
159 )
160
161 volume_fig.update_layout(
162     xaxis_title="Date",
163     yaxis_title="Volume",
164     height=350
165 )
166
167 st.plotly_chart(volume_fig, use_container_width=True)
168
169 # -----
170 # STOCK DATA TABLE
171 # -----
172 st.subheader("📋 Recent Market Data")
173
174 recent_data = data[
175     ["Open", "High", "Low", "Close", "Volume"]
176 ].tail(10).copy()
177
178 recent_data["Open"] = recent_data["Open"].round(2)
179 recent_data["High"] = recent_data["High"].round(2)
180 recent_data["Low"] = recent_data["Low"].round(2)
181 recent_data["Close"] = recent_data["Close"].round(2)
182
183 st.dataframe(
184     recent_data,
185     use_container_width=True
186 )
187

```

Fig 4.10.4 — Trading volume chart and recent market data table (lines 147–187)

Trading volume is rendered as a Plotly bar chart against the same date axis as the price chart, giving a visual sense of activity spikes. Below it, the ten most recent rows of Open/High/Low/Close/Volume data are shown in a rounded, scrollable table via `st.dataframe` for quick numeric reference.

```
app.py > ...
188 # -----
189 # COMPANY INFORMATION
190 # -----
191 st.subheader("📄 Company Information")
192
193 company_col1, company_col2 = st.columns(2)
194
195 company_name = info.get("longName", selected_stock)
196 sector = info.get("sector", "Not Available")
197 industry = info.get("industry", "Not Available")
198 market_cap = info.get("marketCap")
199
200 company_col1.write(f"***Company:** {company_name}")
201 company_col1.write(f"***Sector:** {sector}")
202
203 company_col2.write(f"***Industry:** {industry}")
204
205 if market_cap:
206     company_col2.write(
207         f"***Market Cap:** ${market_cap:,.0f}"
208     )
209 else:
210     company_col2.write("***Market Cap:** Not Available")
211
212 # -----
213 # FOOTER
214 # -----
215 st.divider()
216
217 st.caption(
218     "Stock market data is retrieved using Yahoo Finance through "
219     "the yfinance Python library. Market data may be delayed."
220 )
221
222 except Exception as e:
223     st.error("Unable to retrieve stock market data.")
224     st.info(
225         "Please check your internet connection and try again."
226     )
```

Fig 4.10.5 — Company information section, footer, and error handling (lines 188–226)

Company name, sector, industry, and market capitalization are pulled from the ticker's info dictionary, with a fallback to "Not Available" when a field is missing (e.g., market cap for some tickers). A footer caption credits Yahoo Finance as the data source, and the top-level try/except ensures any retrieval failure is shown as a friendly error message rather than a stack trace.

4.11 Dashboard Screenshots (Output)

The screenshots below show the working Real-Time Stock Market Dashboard for two selected stocks — Apple (AAPL) and NVIDIA (NVDA) — demonstrating the current price metrics, historical price chart, trading volume chart, recent market data table, and company information.

Apple and NVIDIA were chosen for these screenshots specifically because their one-year price trends looked quite different at the time of testing — Apple showing a broadly upward recovery and NVIDIA showing a sharper pullback — which made it easier to visually confirm that the dashboard renders each stock's data independently and correctly rather than reusing cached values across tickers.

Apple (AAPL)



Fig 4.11.1 — Apple: current price metrics and historical price chart

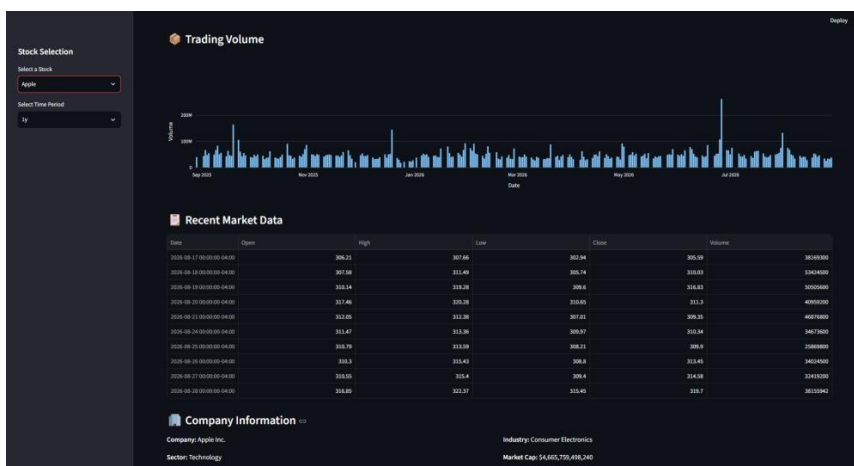


Fig 4.11.2 — Apple: trading volume chart, recent market data table, and company information

At the time of this screenshot, Apple was trading at \$319.70 (+1.63%), with a one-year chart showing a dip through late 2025 followed by a steady recovery into mid-2026 — an example of the kind of trend a user could quickly identify from the historical price chart alone.

NVIDIA (NVDA)

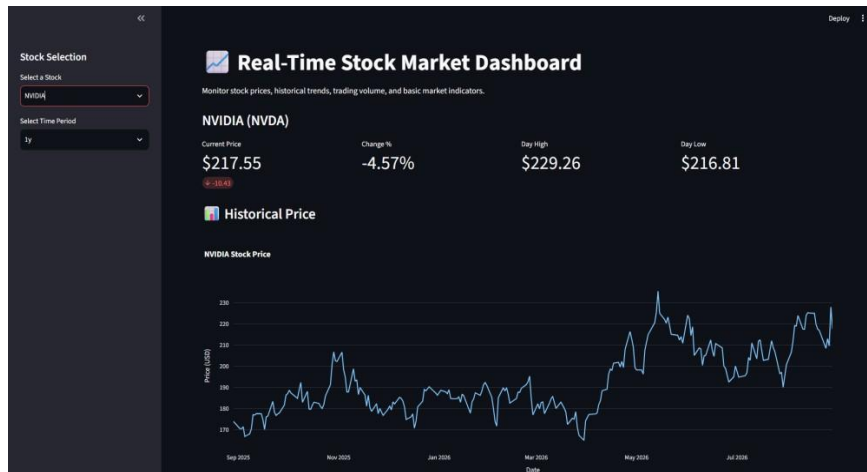


Fig 4.11.3 — NVIDIA: current price metrics and historical price chart

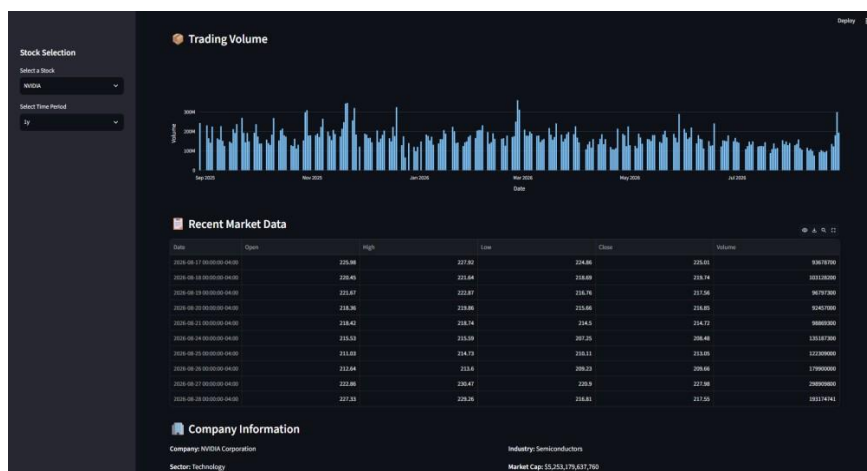


Fig 4.11.4 — NVIDIA: trading volume chart, recent market data table, and company information

NVIDIA's snapshot shows a current price of \$217.55, down 4.57% on the day, with day high/low of \$229.26/\$216.81 — illustrating that the metric cards and colour-coded change indicator update correctly for a stock moving in the opposite direction to Apple in the same session.

4.12 Certificates of Completion and Other Communication / Mail Proof

This section contains the internship offer letter, certificates of completion, and the letter of recommendation issued by Codec Technologies Pvt. Ltd. on successful completion of the internship.

Together, these documents provide an end-to-end paper trail of the internship — from the initial offer and stated duration/designation, through the completion certificates recognized on the AICTE National Internship Portal and by ICAC, to the closing letter of recommendation — supporting the authenticity of the work described in this report.



Fig 4.12.1 — Internship Offer Letter

Letter of Recommendation



Date of Issue - 7/8/2026 [MM/DD/YY]

To Whom It May Concern,

I am writing this letter to highly recommend Aditya Raj for their exceptional performance and unwavering dedication during their tenure as an intern at Codec Technologies. Their time with us has been marked by an exemplary work ethic, a profound passion for the field, and an impressive ability to excel in their assigned tasks.

Throughout their internship, he/she consistently demonstrated a high level of professionalism and initiative, often exceeding our expectations. They possess remarkable technical skills and a strong understanding of the industry, which they applied effectively to tackle complex projects. Their enthusiasm for their work was evident, as they displayed outstanding problem-solving abilities and a keen aptitude for learning.

Based on their outstanding performance and evident potential, I have no doubt that they will continue to thrive and make significant contributions in their future endeavors. It is with great pleasure and confidence that I recommend them for any future opportunities or positions they may seek. Their dedication and capabilities will undoubtedly be a valuable asset to any organization.

Sincerely,

Codec Technologies

Dr. Anurag Shrivastava
Program Manager



www.codectechnologies.in

Powai, Mumbai, Maharashtra 400076

support@codectechnologies.in

Fig 4.12.4 — Letter of Recommendation

4.13 Challenges Faced and Key Learnings

A few practical challenges came up during development. Some stock tickers occasionally returned incomplete metadata (for example, a missing market capitalization figure), which initially caused the company information section to fail; this was resolved by adding fallback values instead of assuming every field would always be present. Repeated API calls on every widget interaction also made the app feel sluggish during testing, which is what motivated adding a time-based cache to the data-loading function. Finally, aligning the four metric cards and two charts consistently across different screen widths required some trial and error with Streamlit's column and container-width options.

These issues, while minor individually, reinforced a broader lesson: real-world data sources are rarely as clean or complete as sample datasets used in coursework, and defensive coding (fallback values, caching, and error handling) is not optional polish but a core part of building anything meant to be used interactively.

4.14 Conclusion

The Real-Time Stock Market Dashboard successfully met the objectives set out at the start of the internship: it retrieves and displays current and historical stock data, visualizes price trends and trading volume interactively, and presents company information alongside these metrics in a single, easy-to-use interface. Beyond the technical deliverable, the internship gave me practical, hands-on experience with the Python data and web-app ecosystem (Streamlit, Pandas, Plotly, yfinance), as well as with the surrounding discipline of testing, version control, and documentation that is expected in a professional development setting. This experience has strengthened my confidence in taking a project from an initial problem statement through to a working, demonstrable product.

5. Bibliography / References

- [1] Python Software Foundation, "Python Language Reference," available at: <https://docs.python.org/3/>
- [2] Streamlit Inc., "Streamlit Documentation," available at: <https://docs.streamlit.io/>
- [3] NumFOCUS, "Pandas Documentation," available at: <https://pandas.pydata.org/docs/>
- [4] Plotly Technologies Inc., "Plotly Python Open Source Graphing Library," available at: <https://plotly.com/python/>
- [5] "yFinance Library Documentation," available at: <https://pypi.org/project/yfinance/>
- [6] Codec Technologies Pvt. Ltd., Internship Guidance and Communication (08/06/2026 to 08/07/2026).
- [7] AICTE National Internship Portal, Ministry of Education, Government of India, available at: <https://internship.aicte-india.org/>
- [8] International Certification Accreditation Council (ICAC), Certificate Verification Portal.